

【九析】第十课：AutoGen 多智能体对话框架——从框架架构到 AgentChat 可视化原型

01

课程主题

项目	内容
课程主题	AutoGen 多智能体对话框架
课时安排	第5周第2次课（2小时）
学员基础	熟悉 Python 编程，理解 Prompt Engineering、Function Calling、ReAct、Agent Skill
教学目标	1. 理解 AutoGen 的技术演进与核心架构设计 2. 掌握 AutoGen v0.4 的三层架构与异步消息机制 3. 深入理解 AgentChat 对话引擎与 ConversableAgent 基类 4. 学会使用 AutoGen Studio 进行可视化多智能体开发 5. 系统对比 AutoGen 与 OpenClaw 的设计哲学与适用场景 6. 完成 AutoGen 开发环境的搭建与验证

02

环境准备

- Python 3.11+ 环境，能正常创建虚拟环境
- 一个可用的大模型 API Key，支持 OpenAI 或 OpenAI 兼容协议
- 可访问 PyPI、GitHub 与 AutoGen 官方文档的网络环境
- 提前安装：`pip install autogen-agentchat autogen-ext[openai]`

03

课程详情

第一部分：开场与回顾

同学们好，欢迎回到 AI Agent 全栈开发课程。

前面几节课，我们完成了单智能体的完整学习：从 Agent 认知革命到第一个实战，从 Function Calling 底层原理到认知框架，并一起完成了一个智能数据分析助手从0到1的构建。学完这些，相信大家已经能够独立开发出功能完善的单智能体 Agent 系统，并且能够在 LangChain 和 OpenClaw 之间做出合理的技术选型了。

但是，我们还有一个重要的问题没有解决，那就是当任务变得足够复杂，一个智能体忙不过来的时候怎么办？现实世界中的复杂任务——比如金融风控系统、软件开发全流程、医疗诊断协作等——通常都需要多个专家分工协作。就像一个人写不完整个操作系统，一个智能体也很难搞定真正复杂的业务。

这就是我们今天要进入的全新阶段——多智能体系统开发。

多智能体系统的核心思想很简单：分工协作，各司其职。把一个大任务拆解成多个子任务，分配给不同的智能体（Agent），每个智能体负责自己擅长的领域，通过标准化协议进行通信和协作，共同完成最终目标。这种架构模仿了人类团队的工作方式——有规划者、执行者、审查者、协调者。

在多智能体框架的版图中，有三个代表性选手：AutoGen、CrewAI和LangGraph。今天我们学习的是微软开源的AutoGen——它是最早提出‘多智能体对话协作’范式的框架，也是目前生产环境采用最广泛的多智能体系统之一。

我们的课程将覆盖以下几个方面：

- 第一部分：AutoGen 是什么？它的技术演进与核心价值。
- 第二部分：AutoGen 核心架构解析——从v0.4的三层架构到异步消息机制。
- 第三部分：AgentChat 机制深度解析——ConversableAgent 与双智能体模式。
- 第四部分：AutoGen Studio——零代码/低代码的可视化开发平台。
- 第五部分：OpenClaw vs AutoGen——框架对比与选型建议。
- 第六部分：环境搭建与首运行——完成安装并跑通第一个双智能体对话。
- 第七部分：总结与作业。

准备好了吗？我们开始。

第二部分：AutoGen是什么？——多智能体协作的开创者

2.1 AutoGen 的诞生与技术演进

AutoGen 是 Microsoft Research 在2023年末发布的开源框架，一个专门用于构建多智能体的协作系统。

它的核心创新是对话式多智能体协作——用‘群聊式协作’替代了单 Prompt 链。

在 AutoGen 出现之前，LLM 的主流用法只有两种：单线程链式调用和简单的工具调用智能体。

AutoGen 带来了一套全新的心智模型：智能体是对话的参与者，整个系统就是一个群聊。智能体之间可以委派任务、互相批评与纠正、调用工具、编写并执行代码、向人类发起询问，在目标达成后自行终止。没有任何一个中央控制器需要提前知晓完整计划，这套流程和人类解决复杂问题的方式高度吻合：分工、讨论、审查、输出。

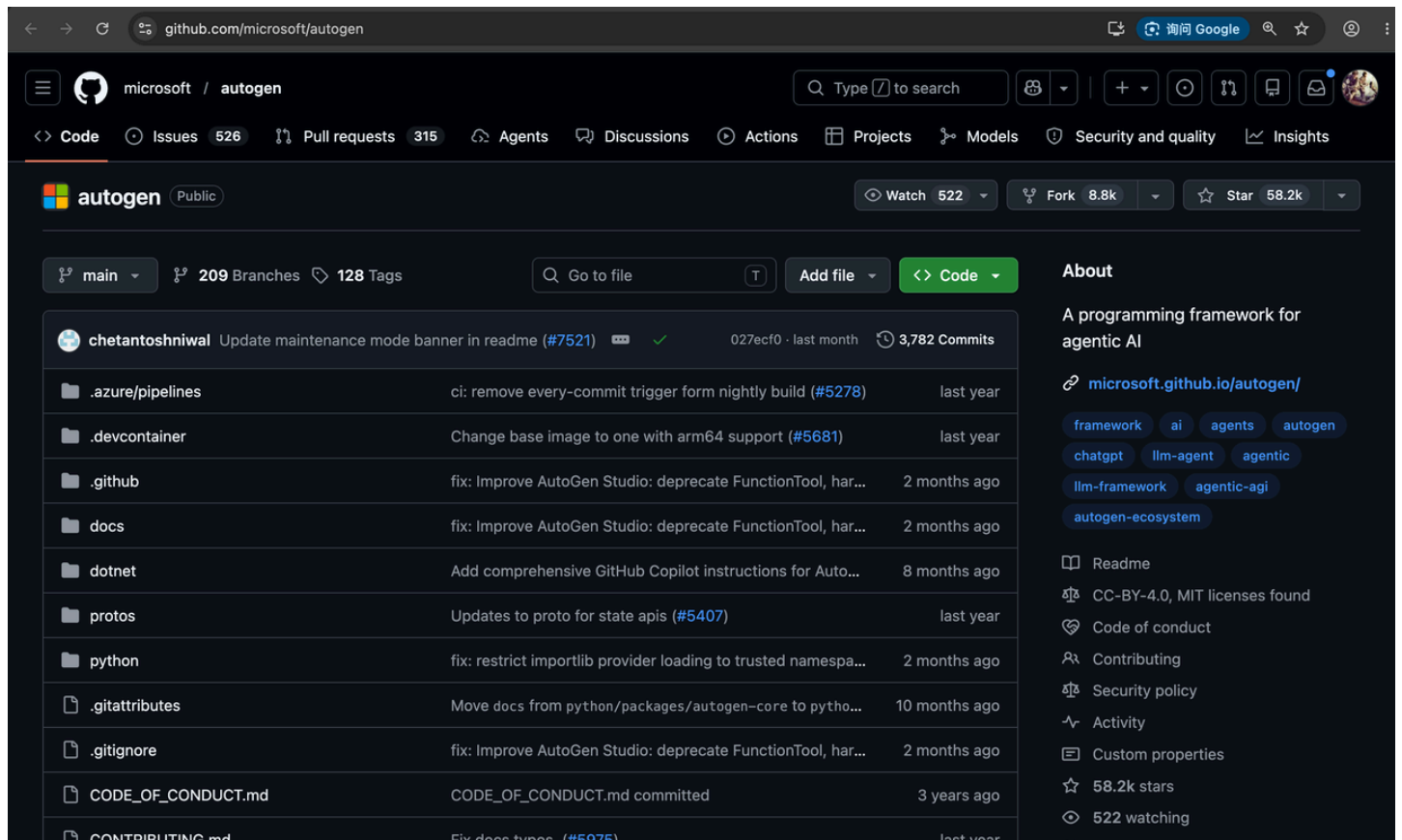
早期的 AutoGen demo 展现了惊人的潜力：编码者+评审者+执行者联合解数学题、网络研究小组、股票分析团队，在许多任务上都展现出比单智能体高 2-10 倍的表现。这些结果验证了多智能体协作的基本原理：分工带来效率，讨论提升质量。

从技术演进过程来看，AutoGen 经历了三次重要迭代：

- 2023年Q4：核心架构公开，确立多智能体对话范式。
- 2024年Q3：v0.2版本引入异步事件驱动机制。
- 2025年Q2：v0.4版本重构底层代码库，达到技术巅峰。

到 2026 年，多智能体协作框架预计将覆盖 60% 以上的AI应用开发场景，成为企业数字化转型的关键基础设施。

[AutoGen Github](#) 官网如下所示：



2.2 AutoGen vs 单智能体：为什么需要多智能体协作？

为什么需要多智能体，而不是继续强化单智能体？上节课我们在介绍多智能体理论时也重点介绍过，根本原因在于任务复杂度的量级跃迁。

单智能体的处理模式是线性的：用户输入→Agent思考→调用工具→输出结果。这个模式在简单任务上表现良好，但当任务要求跨领域知识、需要多人协作时，问题就暴露出来了。举个例子，一个智能体同时充当架构师、程序员、测试员，思维模式单一，容易产生偏见；上下文窗口有限，很难同时处理设计文档、代码、测试报告等多份材料；一旦某个环节卡住，整个任务就停滞了。

AutoGen 的多智能体协作就是为了打破这些限制。它的核心机制可以概括为四个关键词：角色分工、自然语言协商、混合协作以及异步架构。

2.3 AutoGen 在 2026 年的生态定位

2025 年末，微软将 AutoGen 与 Semantic Kernel 合并，统一为 Microsoft Agent Framework (MAF)。v0.4 版本是 AutoGen 独立时期的技术巅峰。虽然当前底层架构已并入 MAF，但大多数人在谈到源自 AutoGen 的多智能体编排风格时，依然习惯说 AutoGen，而非 MAF。此外，AutoGen v0.4 的开源社区仍然活跃，大量生产系统基于 v0.4 构建，存量代码库短期内不会迁移。

根据官方路线图，2025-2026 年的重点突破方向包括：自适应架构、量子计算集成、边缘计算支持。

第三部分：AutoGen 核心架构解析——从 v0.4 到 MAF

3.1 v0.4 的三层架构设计

AutoGen v0.4 本质上相当于 AutoGen 2.0，它用一套全新的三层架构取代了旧版的阻塞式同步 GroupChat。

第一层：autogen-core，这一层是基础设施层，它负责底层事件驱动，它提供 RoutedAgent、订阅机制、发布/订阅消息传递。这一层是所有上层能力的基础，它定义了智能体的核心通信协议。

第二层：autogen-agentchat，这一层是高层 API 层，是大多数人实际使用的部分。它提供很多应用核心组件，开发者在这一层可以定义智能体类型、配置 LLM、设置对话流程。

第三层：autogen-ext，这一层是扩展层，它是可插拔的扩展模块，包含 OpenAI Assistant API 集成、MCP 工作台适配器以及 gRPC 分布式智能体等等。

这套分层架构的核心价值体现在以下三方面：

1. 关注点分离 —— 每一层都只负责自己的职责
2. 模块化扩展 —— 新增功能只需在扩展层添加组件
3. 跨语言支持 —— core 层理论上可支持多种语言，当下 Python 是主力。

3.2 异步消息机制与事件驱动架构

AutoGen v0.4 最大的技术改造是完全异步化。因为传统的同步架构在多 Agent 协作中面临三大挑战：

1. 资源竞争导致的高延迟
2. 顺序依赖引发的单点故障
3. 代码耦合造成的维护困难

AutoGen 0.4 引入了事件驱动架构，并重构 Agent 之间的通信范式，具体包含以下三个核心设计：

1. 第一个设计：引入消息队列中间件。并采用发布-订阅模式解耦生产者和消费者，可支持百万级 QPS 消息吞吐。
2. 第二个设计：引入状态持久化引擎。将 Agent 运行状态存储于分布式键值数据库，确保故障恢复时状态一致性。
3. 第三个设计：智能重试机制。该机制用于处理临时性网络故障，避免消息丢失。

这套异步架构的核心贡献在于：

1. 解耦 Agent 之间直接依赖
2. 单个 Agent 故障不影响整体
3. 高并发支持下，可水平扩展 Agent 数量

从而较完美地应对传统同步架构在多 Agent 协作中面临的上述三大挑战。

3.3 AutoGen vs LangGraph vs CrewAI：架构哲学对比

在深入 AutoGen 实现之前，我们有必要将它放在多智能体框架的坐标系中，将它跟其他多智能体框架——比如：LangGraph、CrewAI 排排坐。

1. LangGraph：图状态机范式。将整个系统建模为有向图，每个节点代表智能体或工具，边表示数据流向，类似线性工作流。所有节点共享全局状态存储，通过图遍历算法实现状态同步。LangGraph 框架适合需要精确控制执行路径的确定性任务，例如：金融风控。
2. CrewAI：角色剧本模型。它引入企业组织架构的隐喻，将智能体视为具有明确角色的员工。通过剧本文件定义角色间的协作流程，每个角色预配置可用的工具集合。CrewAI 在角色分工明确的场景下表现优异。
3. AutoGen：对话协商机制。采用独特的对话驱动架构——智能体通过自然语言消息传递，进行工具调用，它没有中央控制器，通过对话历史实现状态传递。AutoGen 更适合需要动态的开放式协作场景。

简单对比就是：

1. LangGraph 解决‘状态流转’问题
2. CrewAI 解决‘角色分配’问题
3. AutoGen 解决‘自然语言协商’问题

它们解决的其实是不同类问题。

第四部分：AgentChat 机制深度解析

4.1 ConversableAgent 基类设计

上面我们介绍了 AutoGen 的架构设计，知道它包括 `core` 层、`AgentChat` 层和 `ext` 层，下面我们再深入 AgentChat 一探究竟，毕竟它是我们经常需要打交道的地方。AgentChat 基于智能体基类

ConversableAgent 构建而成。

ConversableAgent 是 AutoGen 的智能体基类，它主要实现了三大核心能力：

1. 消息接收与处理管道：它是智能体的通信中枢，负责接收来自其他智能体的消息、解析消息类型、路由到对应的处理器。
2. 提供工具调用标准化接口：它通过 Function Calling 接口对接外部能力。开发者可通过它为智能体添加工具调用能力。
3. 对话状态管理机制：维护智能体的对话历史，支持短长期记忆结合。

为什么要设计 ConversableAgent 作为统一基类？因为它提供了‘共性能力’，比如：消息收发、工具调用、状态管理以及‘扩展接口’等等。

4.2 双智能体模式：AssistantAgent 与 UserProxyAgent

AutoGen 最经典的入门模式是双智能体模式——一个 AssistantAgent（助手智能体）和一个 UserProxyAgent（用户智能体）之间的协作。其中助手智能体负责思考，用户智能体负责执行，也就是一个老板，一个员工。

双智能体的工作流程：

1. UserProxyAgent (user) 收到用户任务 “写一个计算斐波那契数列的函数”
2. UserProxyAgent 将任务消息发送给 AssistantAgent
3. AssistantAgent 生成 Python 代码，在回复中附上代码块
4. UserProxyAgent 收到消息，检测到代码块后自动执行
5. 执行结果发送回 AssistantAgent，形成闭环迭代
6. 当任务完成时，AssistantAgent 返回 `TERMINATE`，UserProxyAgent 结束会话

这套双智能体模式价值在于：UserProxyAgent 承担了“执行者”的角色，而 AssistantAgent 承担了“思考者”的角色，两者形成“想法→行动→反馈→修正”的完美闭环，这是单智能体难以实现的。

4.3 GroupChat：多智能体群聊协调机制

双智能体是 AutoGen 的基础模式，而 AutoGen 真正的威力在于 GroupChat ——让三个及以上智能体在一个群聊中协作。

GroupChat 的核心组件有三个：

1. 角色分配系统：预设角色（如专家、执行者等）。每个智能体在群聊中扮演不同专业角色，形成能力互补。

2. 对话控制策略：包含轮次控制、发言权分配和冲突解决机制。当多个智能体同时发言时，GroupChatManager 决定谁拥有话语权，常用策略有随机选择、轮询和手动指定，也支持根据消息内容智能选择最相关智能体。在医疗诊断场景中可实现“放射科智能体提供影像分析→临床智能体整合病史→主诊智能体生成诊断建议”的协作流程。
3. 上下文管理：维护对话历史并支持上下文窗口动态调整。每个智能体看到的都是整个群聊的历史消息。

GroupChat 的核心价值在于：从线性对话到网状协作。智能体之间不再是简单的点对点通信，而是在群聊中形成多向讨论网络，一个智能体提出的方案可以被其他智能体质疑、修正、补充，可以较大幅度提升输出质量和可靠性。

第五部分：AutoGen Studio——零代码/低代码的可视化开发平台

5.1 Studio 的设计定位与核心价值

AutoGen Studio 是微软开源的低代码/零代码多智能体协作开发平台，通过可视化拖拽界面极大降低了多智能体系统的开发门槛。

它解决了什么问题呢？传统多智能体开发需要编写大量代码来定义智能体、配置工具、协调对话流程，调试多个Agent 之间的消息传递本身就很复杂。AutoGen Studio 让你“像搭积木一样”配置 AI 模型、创建不同角色的智能体、设计它们之间如何协作的工作流，无需深入理解异步编程或 API 调用。

Studio vs 纯手工代码开发的定位差异是：Studio 适合快速原型验证、非技术人员参与、标准化工作流的可视化编排；纯代码开发适合定制化程度高的生产系统、与现有代码库深度集成。

5.2 三层架构与三大核心视图

AutoGen Studio 采用标准的前后端分离架构：

1. 前端交互层：提供可视化工作区与 API 控制台
2. 业务逻辑层：包含智能体编排引擎、任务调度中心与状态管理模块
3. 基础设施层：兼容 Kubernetes 与单机运行模式。

平台提供了三大核心视图：

1. 构建视图（Build View）：拖拽式流程设计器，内置20+预置组件，开发者可通过组件市场下载第三方扩展，支持自定义 Python 函数封装为智能体节点。

2. 游乐场视图（Playground View）：实时调试环境，支持交互式日志追踪可视化展示智能体对话流、动态参数注入、性能分析面板显示各节点响应延迟与资源占用。
3. 画廊视图（Gallery View）：项目模板管理系统，包含12个官方标准模板，以及社区共享的金融风控、医疗诊断等垂直领域方案。

5.3 使用场景与安装方式

AutoGen Studio特别适合三类场景：

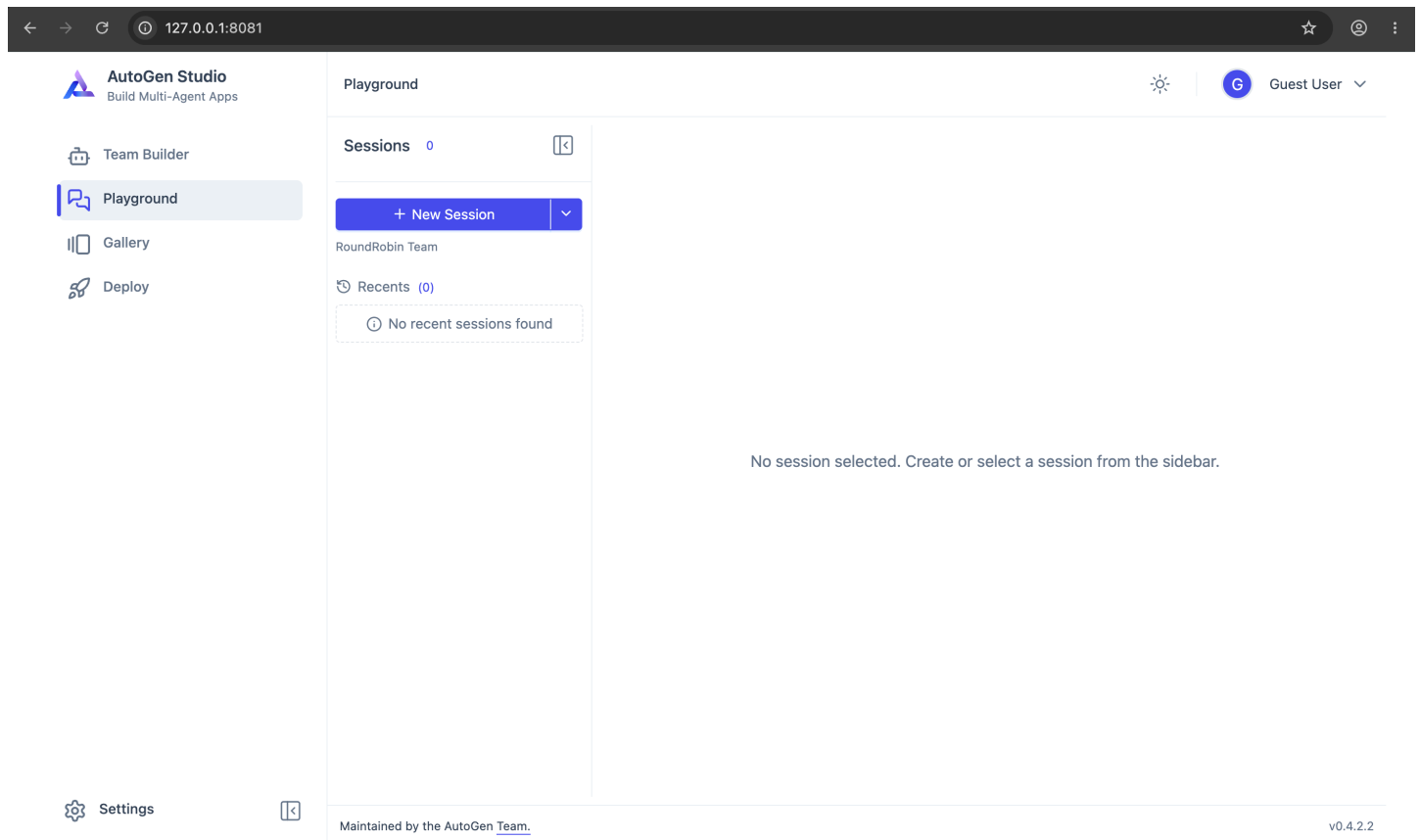
1. 智能客服系统的多轮对话设计
2. 数据分析流程的自动化编排
3. 多模态内容生成工作流的构建

AutoGen Studio 安装与启动：

代码块

```
1 pip install autogenstudio
2 autogenstudio ui
```

默认启动在 8080 端口。如果端口被占用，可用 `autogenstudio ui --port 8081`。启动后效果：



对于生产环境，我们推荐使用容器化方式进行部署。安装后我们就可在浏览器中通过拖拽式界面创建智能体、配置 LLM 模型、设计 workflow。

AutoGen 与 OpenClaw 的技能生态相比，AutoGen Studio 的优势就是可视化编排。

第六部分：OpenClaw vs AutoGen——框架对比与选型建议

6.1 核心理念差异：操作系统 vs 协作框架

OpenClaw 和 AutoGen 代表了两种不同的 Agent 框架范式。

OpenClaw 定位：桌面 Agent 操作系统。它是一个开箱即用的智能体应用，内置 15+ 消息平台接入、子智能体编排、主动行为调度，以及 5700+ 社区技能生态。你可以把它想象成一个‘智能体操作系统’——安装后就能运行，通过技能生态扩展能力，门槛极低。

AutoGen 定位：多智能体协作框架。它是一个 Python 库，需要嵌入到应用中进行开发。它的核心能力是提供智能体间对话协作的编程模型，适用于需要多个 LLM 专家团队协同的复杂场景。

这是两者最根本的区别：OpenClaw 是运行时，开箱即用；AutoGen 是框架，需要开发。

6.2 对比维度：架构设计、核心能力、适用场景

下面我们从四个关键维度深入对比：

第一：部署形态。OpenClaw 是完整的智能体运行时，自带服务端、平台集成；AutoGen 是 Python 库，需嵌入应用开发。

第二：工具/技能扩展。OpenClaw 有 ClawHub 技能市场（5700+技能），技能用 YAML 配置 + Python 实现；AutoGen 通过 Function Calling 机制，需编写 Python 函数并注册到 Agent。

第三：开发门槛与灵活性。OpenClaw：低门槛、配置化、生态丰富，但定制深度受限；AutoGen：高门槛、编程化、高度灵活，适合复杂业务逻辑。

从权威选型建议来看：个人自动化选 OpenClaw；复杂业务逻辑选 AutoGen；企业级研究性工作流选 AutoGen；快速个人研究选 OpenClaw。

还是需要强调下：两种技术方案不是互斥的。实际项目中可以用 OpenClaw 处理外部交互和自动化，用 AutoGen 处理内部多智能体协作推理。

第七部分：环境搭建与首运行

7.1 安装与配置

接下来，我们动手搭建 AutoGen 开发环境。

7.1.1 步骤1：检查 Python 版本

代码块

```
1 python --version # 需要 3.9 或以上，推荐 3.10
```

7.1.2 步骤2：创建虚拟环境

代码块

```
1 python -m venv venv
```

```
2 source venv/bin/activate # Windows: autogen_env\Scripts\activate
```

7.1.3 步骤3：安装核心包

代码块

```
1 pip install -U "autogen-agentchat" "autogen-ext[openai]"
```

7.1.4 步骤4：配置API密钥

创建 `OAI_CONFIG_LIST` 文件，AutoGen的标准配置方式（文件名不可改）：

代码块

```
1  [  
2      {  
3          "model": "gpt-4",  
4          "api_key": "your-openai-api-key"  
5      }  
6  ]
```

7.1.5 步骤5：验证安装

代码块

```
1 python -c "import autogen; print(autogen.version)"
```

7.2 第一个AutoGen程序：双智能体对话

验证安装后，我们运行第一个 AutoGen 程序。

代码块

```
1 import json
2 import autogen
3
4 # 从 JSON 配置文件加载模型配置
5 with open("OAI_CONFIG_LIST", "r") as f:
6     config_list = json.load(f)
7
8 # 创建 AssistantAgent
9 assistant = autogen.AssistantAgent(
10     name="helpful_engineer",
11     llm_config={"config_list": config_list},
12     system_message="You are a senior Python engineer. Write clean, efficient code."
13 )
14
15 # 创建 UserProxyAgent
16 user_proxy = autogen.UserProxyAgent(
17     name="user",
18     human_input_mode="NEVER",
19     max_consecutive_auto_reply=10,
20     code_execution_config={"work_dir": "coding", "use_docker": False}
21 )
22
23 # 发起对话
24 user_proxy.initiate_chat(
25     assistant,
26     message="Write a Python function to calculate the nth Fibonacci number, with type hints."
27 )
```

代码执行成功后的日志片段如下：

代码块

```
1 (venv) ai@aideMacBook autogen % python autogen_da.py
2 user (to helpful_engineer):
3
4 Write a Python function to calculate the nth Fibonacci number, with type hints.
5
6 -----
7 helpful_engineer (to user):
8
9 Sure, here is a Python function using recursion to calculate the nth Fibonacci number:
```

```

10
11 ```python
12 def fibonacci(n: int) -> int:
13     if n <= 0:
14         raise ValueError("n should be a positive integer")
15     elif n == 1:
16         return 0
17     elif n == 2:
18         return 1
19     else:
20         return fibonacci(n-1) + fibonacci(n-2)
21 ```

```

通过解读日志，我们知道：AssistantAgent 根据 UserProxyAgent 发送给它的任务生成代码、UserProxyAgent 自动执行代码并返回结果、AssistantAgent 根据执行结果进行修正迭代、任务完成后自动终止。

7.3 添加工具调用功能

最后我们演示下如何为 AutoGen 智能体添加工具调用能力。

代码块

```

1  from autogen import ConversableAgent, register_function
2  import json
3  import requests
4
5  # 加载配置
6  with open("OAI_CONFIG_LIST", "r") as f:
7      config_list = json.load(f)
8
9  def get_weather(city: str) -> str:
10     """获取指定城市的天气信息"""
11     weather_db = {"北京": "晴 5-15°C", "上海": "阴 8-18°C"}
12     return weather_db.get(city, "未知城市")
13
14  # 注册工具函数
15  assistant = ConversableAgent(name="assistant", llm_config={"config_list":
16  config_list})
17  user_proxy = ConversableAgent(name="user")
18
19  register_function(
20      get_weather,

```

```
20     caller=assistant,
21     executor=user_proxy,
22     description="获取城市天气"
23 )
```

执行结果我们发现给 AutoGen 添加工具非常方便。

上面的代码还有很大的优化空间，比如控制 user 智能体如果向 assistant 智能体发送空消息处理等，大家可以自己在此基础上进行优化。

第八部分：课程总结与作业

8.1 核心要点回顾

今天的课程内容非常丰富，我们来快速回顾核心要点：

知识点层面：

- ☒ AutoGen演进历程：从2023年开源到v0.4异步架构，再到并入MAF。
- ☒ 三层架构：core层提供事件驱动原语，agentchat层提供高层API，ext层提供扩展。
- ☒ 异步消息机制：事件驱动架构使系统吞吐量提升40%，响应时间缩短至300ms。
- ☒ AgentChat机制：ConversableAgent基类、AssistantAgent与UserProxyAgent双智能体模式、GroupChat群聊协调。
- ☒ AutoGen Studio：零代码可视化开发平台，三大核心视图降低多智能体开发门槛。
- ☒ OpenClaw vs AutoGen：运行时 vs 开发框架的核心差异，基于场景的选型决策。

能力层面：

- ☒ 能够理解AutoGen的技术演进与核心架构
- ☒ 能够用ConversableAgent构建双智能体对话系统
- ☒ 能够为智能体添加工具调用能力
- ☒ 能够完成AutoGen环境搭建并运行第一个程序
- ☒ 能够根据场景做出AutoGen vs OpenClaw的选型决策

8.2 课后作业

必做作业：

1. 完成 AutoGen 环境搭建，配置 OpenAI API 或国产模型 API，运行双智能体对话示例。
2. 阅读 AutoGen 官方文档中关于 GroupChat 的部分，总结三种发言选择策略的适用场景。

04

附录：教学资源

推荐阅读

- [AutoGen官方文档](#)
- [AutoGen GitHub仓库](#)
- [AutoGen Studio文档](#)